

Java abstract classes and interfaces

What abstract classes and interfaces are, how they differ, when to pick each, and how interfaces buy us loose coupling and lambdas. Definitions stay tight; the answer does the teaching.

Contents

1. [What is an abstract class?](#)
2. [What is an interface?](#)
3. [Abstract class v/s interface](#)
4. [When should you use an abstract class?](#)
5. [When should you use an interface?](#)
6. [Can an abstract class implement multiple interfaces?](#)
7. [Can one interface extend multiple interfaces?](#)
8. [How do interfaces support loose coupling?](#)
9. [What is a functional interface, and why was @FunctionalInterface introduced?](#)

1. What is an abstract class?

An abstract class is a class that **cannot be instantiated directly**. It exists to be extended by other classes. It can hold both abstract methods (no body) and concrete methods (with an implementation).

```
abstract class Animal {
    // abstract method
    abstract void sound();

    // concrete method
    void eat() {
        System.out.println("Eating...");
    }
}

class Dog extends Animal {
    @Override
    void sound() {
        System.out.println("Bark");
    }
}
```

Using it:

```
public class Main {
    public static void main(String[] args) {
        Animal animal = new Dog();

        animal.sound(); // Bark
        animal.eat();   // Eating...
    }
}
```

2. What is an interface?

An interface is a **contract for classes**. It says what a class must do, not how it does it. Any class that implements the interface has to provide the bodies for its methods.

```
interface Payment {
    void pay(double amount);
}

class CreditCardPayment implements Payment {
    @Override
    public void pay(double amount) {
        System.out.println("Paid using credit card");
    }
}

class UPIPayment implements Payment {
    @Override
    public void pay(double amount) {
        System.out.println("Paid using UPI");
    }
}
```

3. Abstract class v/s interface.

Feature	Abstract class	Interface
Purpose	Share common implementation and state	Define a contract
Instantiation	No	No
Constructors	Yes	No
Instance variables	Yes	No (only public static final constants)
Abstract methods	Yes	Yes
Concrete methods	Yes	Yes (default, static, private)
Access modifiers	Any (private, protected, public)	Methods are public by default
Inheritance	A class can extend one abstract class	A class can implement many interfaces
Relationship	IS-A	CAN-DO, a capability
Best use	Share code among closely related classes	Abstraction and loose coupling

4. When should you use an abstract class?

Use an abstract class when related classes share common state or implementation. So reach for it when:

- The classes have an IS-A relationship.
- We want to share common code.
- We need instance variables (state).

- We need constructors.
- Some methods should be common, while others are implemented differently by each subclass.

```
abstract class Animal {  
    protected String name;  
  
    Animal(String name) {  
        this.name = name;  
    }  
  
    void eat() {  
        System.out.println("Eating...");  
    }  
  
    abstract void sound();  
}
```

Every animal eats the same way and has a name, but each makes a different sound. The abstract class holds the shared logic once, so the subclasses do not duplicate it.

5. When should you use an interface?

Use an interface when we want to define a contract without pinning down the implementation. So reach for it when:

- Unrelated classes need to expose the same behaviour.
- We want loose coupling.
- We expect several implementations.
- We want to support dependency injection and mocking.
- A class may need to take on more than one capability.

```
interface Payment {  
    void pay(double amount);  
}
```

The implementations:

```
class UPIPayment implements Payment { }  
class CreditCardPayment implements Payment { }  
class PayPalPayment implements Payment { }
```

The caller depends only on the contract, never on a concrete class:

```
Payment payment = new UPIPayment();
```

6. Can an abstract class implement multiple interfaces?

Yes. An abstract class can implement as many interfaces as it needs.

```

interface Flyable {
    void fly();
}

interface Swimmable {
    void swim();
}

abstract class Animal implements Flyable, Swimmable {
    void eat() {
        System.out.println("Eating...");
    }
}

```

Note that it does not have to provide the bodies for `fly()` and `swim()`. Being abstract, it can leave them for its subclasses to implement.

7. Can one interface extend multiple interfaces?

Yes. Unlike a class, which can extend only one class, an interface can inherit from several interfaces at once.

```

interface Flyable {
    void fly();
}

interface Swimmable {
    void swim();
}

interface Duck extends Flyable, Swimmable {
    void quack();
}

```

Any class that implements `Duck` now has to implement all three methods.

```

class MallardDuck implements Duck {
    @Override
    public void fly() {
        System.out.println("Flying");
    }

    @Override
    public void swim() {
        System.out.println("Swimming");
    }

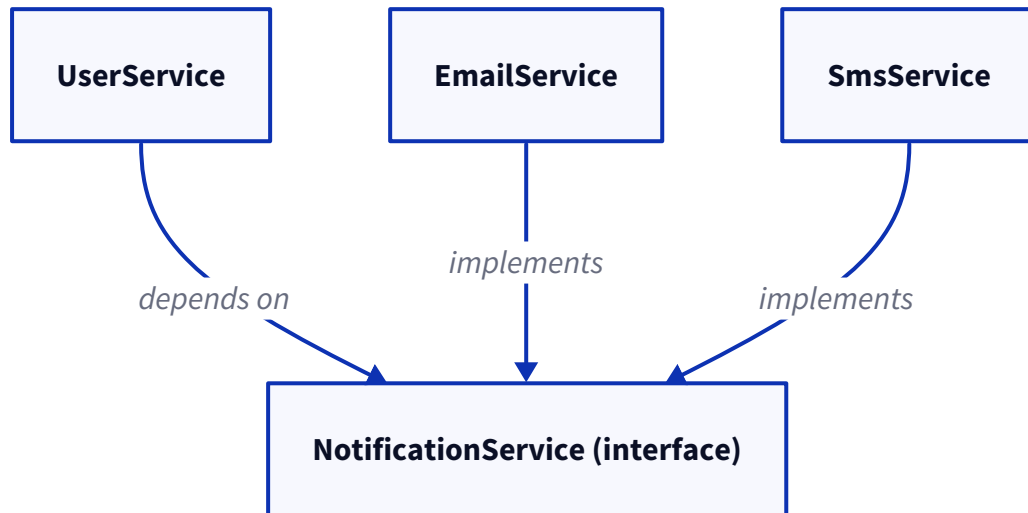
    @Override
    public void quack() {
        System.out.println("Quack");
    }
}

```

8. How do interfaces support loose coupling?

An interface lets a class depend on an **abstraction** instead of a concrete implementation. Once the dependency is on the interface, we can swap the implementation without touching the class that uses it.

The idea is that a class should know **what** another class can do, not **how** it does it. So instead of `UserService` reaching straight for `EmailService`, it depends on a `NotificationService` interface, and the concrete senders implement that interface.



Now `UserService` does not care which implementation it gets. Swapping email for SMS, or dropping in a fake for a test, needs no change to `UserService` at all. That is the payoff of coding to the interface.

9. What is a functional interface, and why was `@FunctionalInterface` introduced?

A functional interface is an interface with **exactly one abstract method** (the SAM, single abstract method). It is the target type for lambda expressions and method references, and it arrived with Java 8.

```
@FunctionalInterface
interface Calculator {
    int add(int a, int b);
}
```

Used with a lambda:

```
Calculator calculator = (a, b) -> a + b;
System.out.println(calculator.add(10, 20));
```

The rules: a functional interface

- must have exactly one abstract method,
- can have any number of `default` methods,
- can have any number of `static` methods,
- can override `Object` methods (`equals()`, `hashCode()`, `toString()`), and those do not count as the abstract method.

```
@FunctionalInterface
interface Printer {
    void print();

    default void log() {
        System.out.println("Logging...");
    }

    static void version() {
        System.out.println("1.0");
    }
}
```

This is still a functional interface, because `print()` is the only abstract method.

Why the annotation: strictly speaking it is optional. This compiles and is still a functional interface without it:

```
interface Calculator {
    int add(int a, int b);
}
```

What `@FunctionalInterface` buys us is compile-time protection. If someone later adds a second abstract method, the compiler rejects it straight away:

```
Unexpected @FunctionalInterface annotation
Calculator is not a functional interface
```

Without the annotation, that same change would compile fine, and the interface would quietly stop working with lambdas. The annotation turns a silent breakage into a clear compile error.